



Comparativa v1 vs v2 vs v3 vs v4

Resumen Rápido

	v1	v2	v3	v4
Subtítulo	Nested JSON	JSON Separados	Carga Asincróna	Lazy Loading
Estructura	Árbol anidado	3 JSONs + lookups	3 JSONs + lookups async	Caché dinámico
Performance	△ O(n)	✓ O(1)	✓ O(1)	✓ O(1)
Carga datos	Import sync	Import sync	fetch() async full	fetch() on-demand
useEffect	✗ No	✗ No	✓ 1 efect	✓ 3 effects
Caché dinámico	✗	✗	✗	✓
Lazy loading	✗	✗	✗	✓
Carga inicial	Instant	Instant	50ms loading	5ms ✓
Ancho banda init	500 KB	390 KB	390 KB	5 KB ✓
Realismo	Educativo	Mejor	Muy realista	★ Profesional
Casos de uso	Aprender	Optimización	Producción	Producción escalable

Detalles Técnicos

● v1: Nested JSON (Estructura Jerárquica)

Características:

- Un único archivo `arbol.json` con estructura anidada
- Comunidades contienen provincias, provincias contienen ciudades
- Búsquedas lineales O(n) para cada nivel

Estructura de datos:

```
[
  {
    "code": "01",
    "label": "Andalucía",
    "provinces": [
```

```

    {
      "code": "04",
      "label": "Almería",
      "towns": [
        { "code": "3", "label": "Agurain/Salvatierra" },
        ...
      ]
    },
    ...
  ]
},
...
]

```

Ventajas:

-  Intuitivo (espejo de la estructura real)
-  Fácil de entender para principiantes
-  Datos relacionados juntos

Desventajas:

-  Búsquedas lentas ($O(n)$)
-  Duplicación de datos
-  Más pesado en memoria

Código de acceso:

```

const provinces = community.provinces.find(p => p.code ===
code)?.provinces || []

```

 v2: Normalized JSON (Estructuras Separadas)**Características:**

- 3 archivos JSON separados: `ccaa.json`, `provincias.json`, `poblaciones.json`
- Cada entidad es independiente
- Relaciones via `parent_code`
- Lookups pre-computados en memoria al cargar

Estructura de datos:

```

ccaa.json:
[
  { "code": "01", "label": "Andalucía", "parent_code": "0" },
  ...
]

```

```
provincias.json:
[
  { "code": "04", "label": "Almería", "parent_code": "01" },
  ...
]

poblaciones.json:
[
  { "code": "3", "label": "Agurain/Salvatierra", "parent_code": "04" },
  ...
]
```

Ventajas:

-  Búsquedas O(1) con lookups
-  22% menos tamaño
-  37% menos consumo memoria
-  Separación de responsabilidades
-  Escalable (fácil agregar más datos)

Desventajas:

-  Más complejo de entender inicialmente
-  Requiere lookups pre-computados
-  Datos cargados en memoria

Código de acceso:

```
const PROVINCES_BY_COMMUNITY: { [key: string]: Province[] } = {}
provinciasData.forEach(p => {
  if (!PROVINCES_BY_COMMUNITY[p.parent_code]) {
    PROVINCES_BY_COMMUNITY[p.parent_code] = []
  }
  PROVINCES_BY_COMMUNITY[p.parent_code].push(p)
})

const provinces = PROVINCES_BY_COMMUNITY[selectedCommunity] || []
```

v3: Async Fetch (Carga Asincrónica)

Características:

- Misma estructura normalizada que v2
- **Pero:** Datos cargados ASINCRONAMENTE con `fetch()`
- `useEffect` para manejar el ciclo de vida
- Estados explícitos para loading, error, data

Estructura de datos:

```
const [data, setData] = useState({
  communities: [],
  provinces: [],
  towns: [],
  loading: true,
  error: null,
})
```

Ventajas:

-  Realista (como en aplicaciones reales)
-  Manejo explícito de loading
-  Manejo explícito de errores
-  Fácil cambiar a API backend
-  O(1) búsquedas
-  Enseña `useEffect` y `fetch()`

Desventajas:

-  Más complejo
-  Latencia inicial (aunque mínima para JSONs locales)
-  Estados adicionales para manejar

Código de carga:

```
useEffect(() => {
  const loadData = async () => {
    const [ccaaRes, provinciasRes, poblacionesRes] = await Promise.all([
      fetch('./src/data/ccaa.json'),
      fetch('./src/data/provincias.json'),
      fetch('./src/data/poblaciones.json'),
    ])

    const communities = await ccaaRes.json()
    const provinces = await provinciasRes.json()
    const towns = await poblacionesRes.json()

    // Pre-compute lookups
    // Remove no ha desaparecido, ahora está en el useEffect

    setData({
      communities,
      provinces,
      towns,
      loading: false,
      error: null,
    })
  }
})
```

```
loadData()  
, [])
```

Progresión de Aprendizaje

v1 → v2 → v3

v1: Aprenderá qué es React, useState, select dependientes

↓

v2: Aprenderá optimización, patrones normalizados, lookups O(1)

↓

v3: Aprenderá a cargar datos reales, useEffect, fetch, async/await

Evolución del Código de Carga

v1 (Import directo)

```
import arbolData from '../data/arbol.json' // Sincrónico  
  
export default function ProvinceSelector() {  
  const communities = arbolData // Disponible inmediatamente  
}
```

v2 (Import de múltiples archivos)

```
import ccaaData from '../data/ccaa.json'  
import provinciasData from '../data/provincias.json'  
import poblacionesData from '../data/poblaciones.json'  
  
export default function ProvinceSelector() {  
  // Pre-compute lookups al cargar el módulo  
  const PROVINCES_BY_COMMUNITY = {}  
  provinciasData.forEach(p => { ... })  
  
  const provinces = PROVINCES_BY_COMMUNITY[selectedCommunity] || []  
}
```

v3 (Fetch asíncronico)

```
export default function ProvinceSelector() {  
  const [data, setData] = useState({ loading: true, ... })
```

```

useEffect(() => {
  const loadData = async () => {
    const ccaaRes = await fetch('./src/data/ccaa.json')
    const ccaaData = await ccaaRes.json()
    // ... más código
  }
  loadData()
}, [])

if (data.loading) return <div>Cargando...</div>

const provinces = data.provincesByCode[selectedCommunity] || []
}

```

Cuándo usar cada versión

Usa v1 si:

- Estás empezando con React
- Necesitas entender selectores dependientes
- El dataset es muy pequeño y simple

Usa v2 si:

- Necesitas optimización de performance
- Trabajas con datasets pequeños-medianos
- Los datos son estáticos (pre-computados)

Usa v3 si:

- Aprendes conceptos avanzados (useEffect, fetch)
- Necesitas cargar datos desde un servidor
- Quieres una arquitectura realista de producción
- Necesitas manejar estados de carga y error

Usa v4 si:

- Escalabilidad es crítica
- Dataset es mediano-grande (>1MB)
- Necesitas optimización de memoria y ancho de banda
- Usuarios típicamente solo usan parte de los datos
- Quieres patrón profesional usado en empresas grandes



Benchmarks Simulados

(Asumiendo 10,000 ciudades y 100 búsquedas)

Operación	v1	v2	v3	v4
-----------	----	----	----	----

Operación	v1	v2	v3	v4
Carga inicial	~5ms	~5ms	50ms (loading)	5ms 
Búsqueda provincia	~50ms (O(n))	<1ms (O(1))	<1ms (O(1))	<1ms cached
Búsqueda ciudad	~100ms (O(n²))	<1ms (O(1))	<1ms (O(1))	<1ms cached
Prov on-demand	N/A	N/A	N/A	30ms 
Tamaño JSON	500 KB	390 KB	390 KB	5KB init 
Mem inicial	10 MB	6.3 MB	6.3 MB	0.5 MB 
Total ancho banda	500 KB	390 KB	390 KB	35-390 KB*

*v4: Depende del uso (5KB si ignora, 390KB si explora todo)

Lecciones por versión

v1 enseña:

- useState hook basics
- Selects dependientes
- Array.find() y .map()

v2 enseña:

- Normalización de datos
- Diccionarios/lookups
- Performance (O(n) vs O(1))
- JavaScript Map/Dict patterns

v3 enseña:

- useEffect hook
- fetch() API
- async/await
- Promise.all()
- Manejo de estados asincronos
- Patrones reales de aplicaciones web

v4 enseña:

- Múltiples useEffect hooks coordinados
- Caché dinámico indexado
- Lazy loading / on-demand loading
- Verificación de caché antes de fetch
- Dependencias complejas de useEffect
- Patrones profesionales de optimización

❓ v4: Lazy Loading (Carga Bajo Demanda)

Características principales:

- Carga comunidades al iniciar (~5ms)
- Carga provincias cuando el usuario selecciona comunidad
- Carga ciudades cuando el usuario selecciona provincia
- Caché dinámico evita re-descargas

Por qué v4 es mejor para datos grandes:

```
// v3: Descarga TODO aunque el usuario no lo use
Promise.all([
  fetch(ccaa),          // 5 KB
  fetch(provincias),    // 50 KB
  fetch(ciudades)       // 335 KB ← Usuario podría ignorar
]) // Total: 390 KB descargados

// v4: Descarga solo lo necesario
fetch(ccaa) // 5 KB – usuario inmediato
// Usuario selecciona Andalucía
fetch(provincias).filter(p => p.parent_code === 'AND') // 10 KB
// Usuario selecciona Almería
fetch(ciudades).filter(c => c.parent_code === 'ALM') // 20 KB
// Total: Solo 35 KB si explora 1 comunidad
```

Patrón de caché:

```
// Después de explorar Andalucía + Almería
data.provinces = {
  'AND': [...provincias de Andalucía...]
}
data.towns = {
  'ALM': [...ciudades de Almería...]
}

// Si usuario vuelve a Andalucía
if (data.provinces['AND']) { // ← Found in cache!
  return // No hace fetch, usa caché
}
```

Ventajas reales:

- ✅ 10x más rápido en inicio (5ms vs 50ms)
- ✅ 78x menos ancho de banda inicial
- ✅ Patrón usado en LinkedIn, Google Maps, Amazon
- ✅ Escalable a millones de registros
- ✅ Mejor UX: UI responsiva inmediatamente

Salto a Producción

Para convertir v3 o v4 a una aplicación de producción real:



Resumen: Cuál Elegir

Situación	Elige
Estoy aprendiendo React	v1
Necesito optimizar $O(n) \rightarrow O(1)$	v2
Cargaré datos de un servidor	v3
Escalabilidad es crítica	v4
Dataset es ENORME (>10MB)	v4 + API paginated

¡Cada versión es un paso importante hacia aplicaciones React profesionales! 🎯

Ver también: [COMPARATIVA_EFICIENCIA.md](#) para análisis técnico detallado de v3 vs v4